What we're building is essentially a Client Relationship & Project Management Portal integrated into our agency's website and CMS.

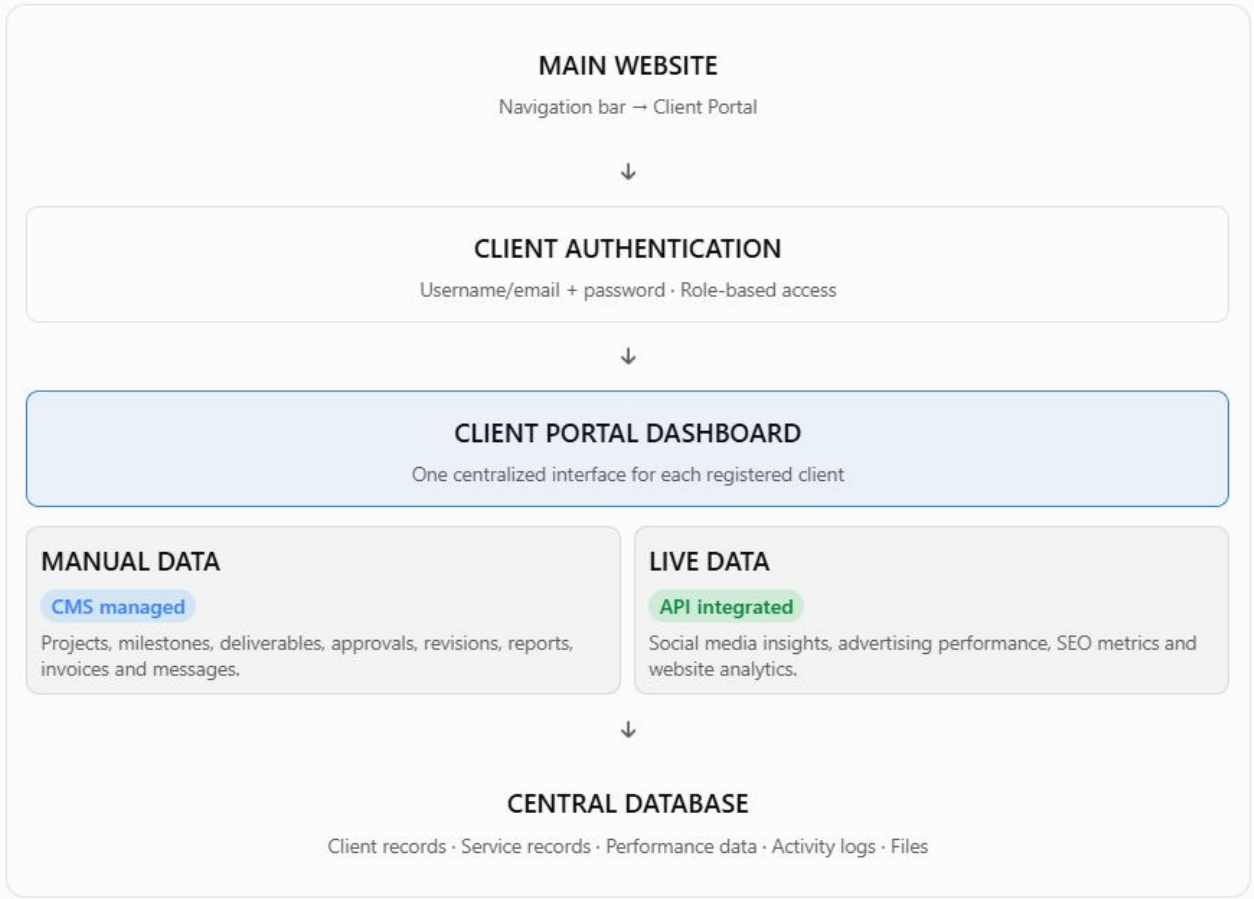
The most important thing to communicate to our developer is that this should not be a single dashboard where every service is updated in the same way.

We need a hybrid data architecture:

1. Manual updates through our CMS for services where our team controls the workflow and deliverables.
2. Automated API integrations for services where performance data is generated by external platforms such as Meta, Google Ads, Google Analytics and Google Search Console.
3. A centralized client portal that combines both types of information into one dashboard.

This distinction is the foundation of the backend logic.

1. Overall system architecture



Note:

The client should never need to know whether information was entered manually or fetched through an API. Everything should appear seamlessly within the same portal.

2. The seven services: how backend updates should work

Service	Backend update mechanism	What the client sees
1. Branding	CMS manual updates	Strategy, brand guidelines, logo concepts, design submissions, milestones
2. Website Development	CMS manual updates + optional technical monitoring	Development stages, preview links, testing, launch status
3. PPC & SEO	API integration	Ad spend, impressions, clicks, conversions, CPC, SEO rankings and organic traffic
4. Social Media Marketing	API integration	Followers, reach, impressions, engagement, interactions and content performance
5. Digital Campaign	Hybrid	Campaign milestones manually updated + live ad performance through APIs
6. Creative Production	CMS manual updates	Shoot schedules, production progress, uploaded photos/videos, deliverables and approvals
7. Influencer Marketing	CMS manual updates + optional platform integrations	Influencer details, campaign progress, content links, views, reach and engagement

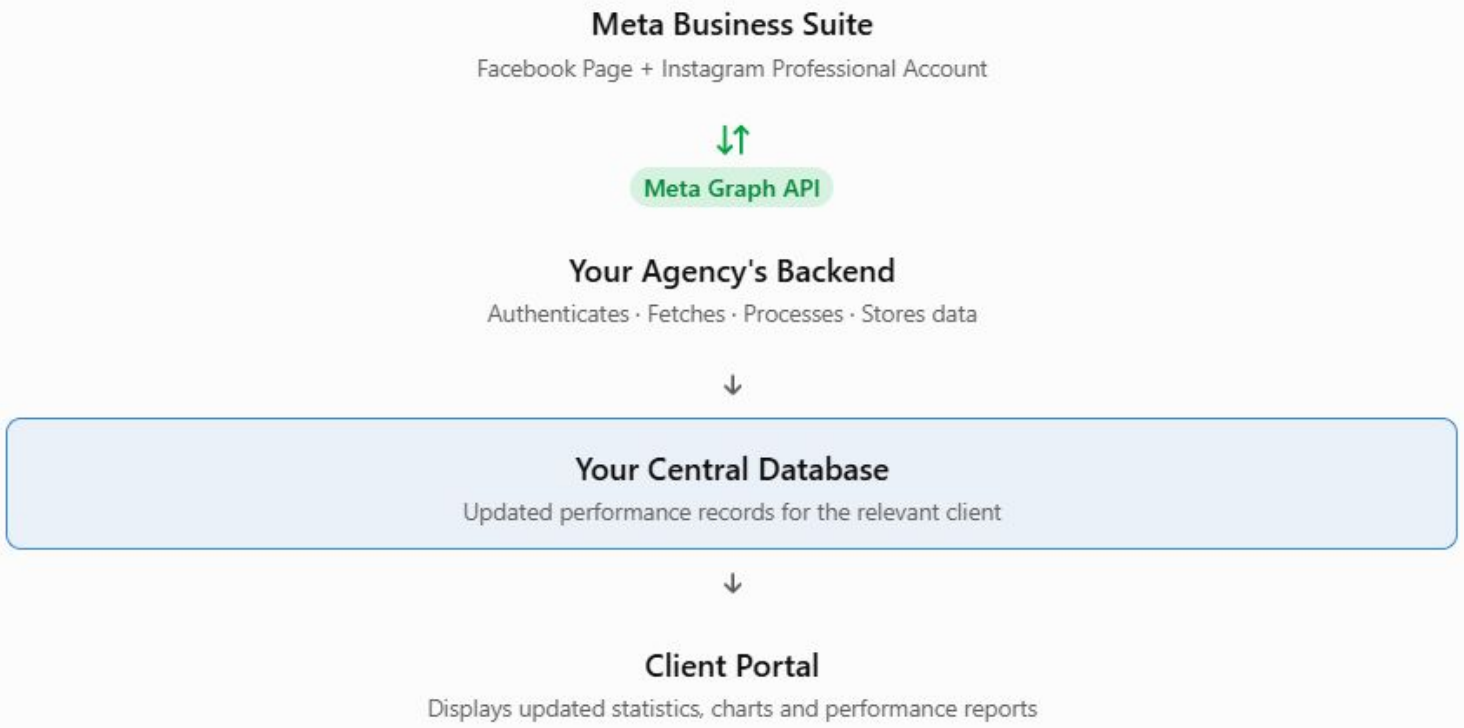
3. How real-time data fetching actually works

This is where our developer needs to make the right technical decision.

Our CMS cannot independently generate live social media or advertising statistics. It needs to communicate with the platforms through their official APIs.

An API (Application Programming Interface) allows our backend to request information from another platform, subject to that platform's permissions and access rules.

Example: Social Media Marketing client



The process should work as follows:

Step 1 — Connect the client's accounts

When onboarding a client, our agency connects the relevant business accounts to our backend.

For example:

- Facebook Page
- Instagram Professional Account
- Google Ads account
- Google Analytics 4 property
- Google Search Console property

This should be done through secure authorization (OAuth where supported), with the necessary permissions granted by the account owner.

Step 2 — Store API credentials securely

The backend stores the access tokens and account identifiers securely.

The developer must ensure that:

- Access tokens are encrypted.
- Tokens are never exposed in frontend code.
- Tokens are refreshed when required.
- Expired or revoked access is detected.
- Each client's accounts are mapped to the correct client record.

Step 3 — Fetch data automatically

The backend requests performance information from the connected platforms.

Platform	Data to fetch
Meta Graph API	Reach, impressions, engagement, followers and content insights
Google Ads API	Spend, impressions, clicks, CTR, CPC and conversions
Google Analytics Data API	Users, sessions, traffic sources, engagement and conversions
Google Search Console API	Clicks, impressions, CTR, average search position and search queries

Step 4 — Save the fetched data in our database

This is important.

Do not make the client dashboard directly request data from Meta or Google every time a client opens the portal.

Instead, your backend should fetch the data, process it and store it in our own database.

This provides:

- Faster dashboard loading.
- Historical performance tracking.
- Consistent reporting.
- Reduced API requests.
- Better control over data access.

Step 5 — Display the updated information

The frontend dashboard reads from our database and displays the latest available information.

The client sees the numbers, charts and reports without having to log into Meta Business Suite or Google Ads separately.

4. How frequently should the data update?

One important clarification: API-powered data is not necessarily instant. The actual update frequency depends on the platform, API limits, data availability and permissions.

recommendation: A scheduled synchronization system (also called a background job or cron job).

Suggested configuration:

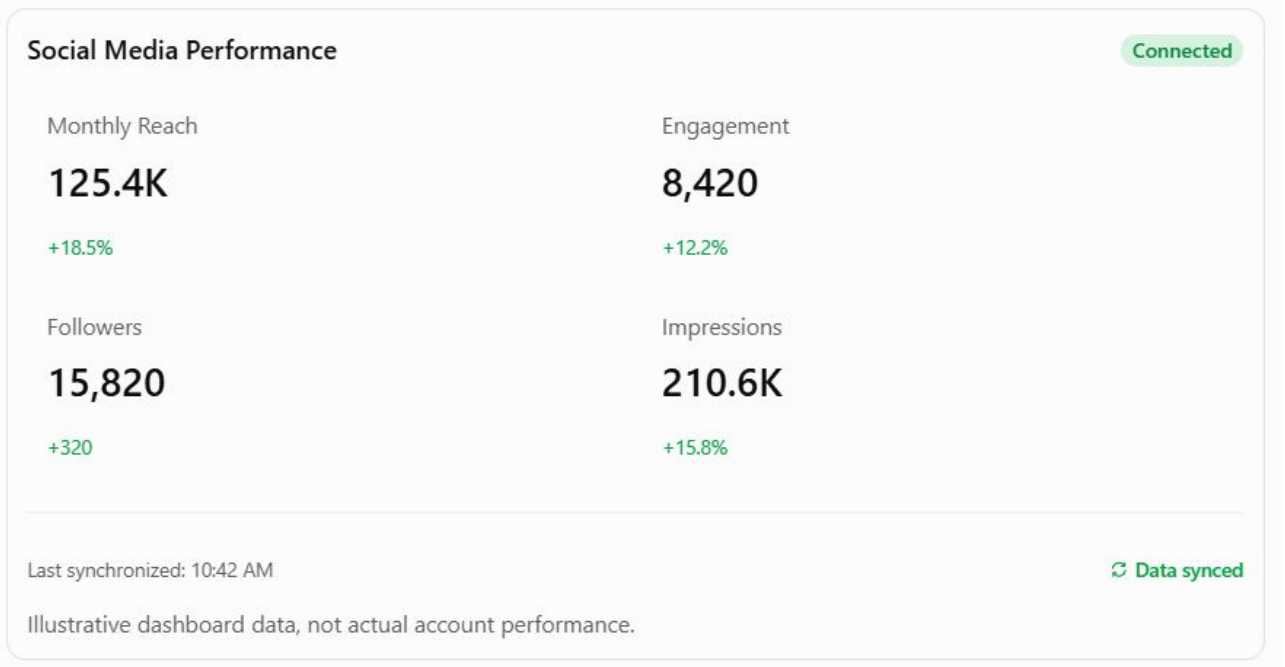
Data source	Suggested synchronization
Meta social media insights	Every 1–3 hours
Google Ads	Every 1–3 hours
Google Analytics 4	Every 1–3 hours
Google Search Console	Once or twice daily
Manual CMS updates	Immediately after publishing

Note:
These are proposed operationa
not guaranteed platform refres

The developer can implement:

- **Scheduled polling:** The backend automatically checks platforms at defined intervals.
- **Webhooks:** Where supported, platforms notify your backend when specific events occur.
- **Manual sync:** An admin can trigger a refresh when needed.
- **Last updated timestamp:** Every performance dashboard should show when its data was last successfully synchronized.

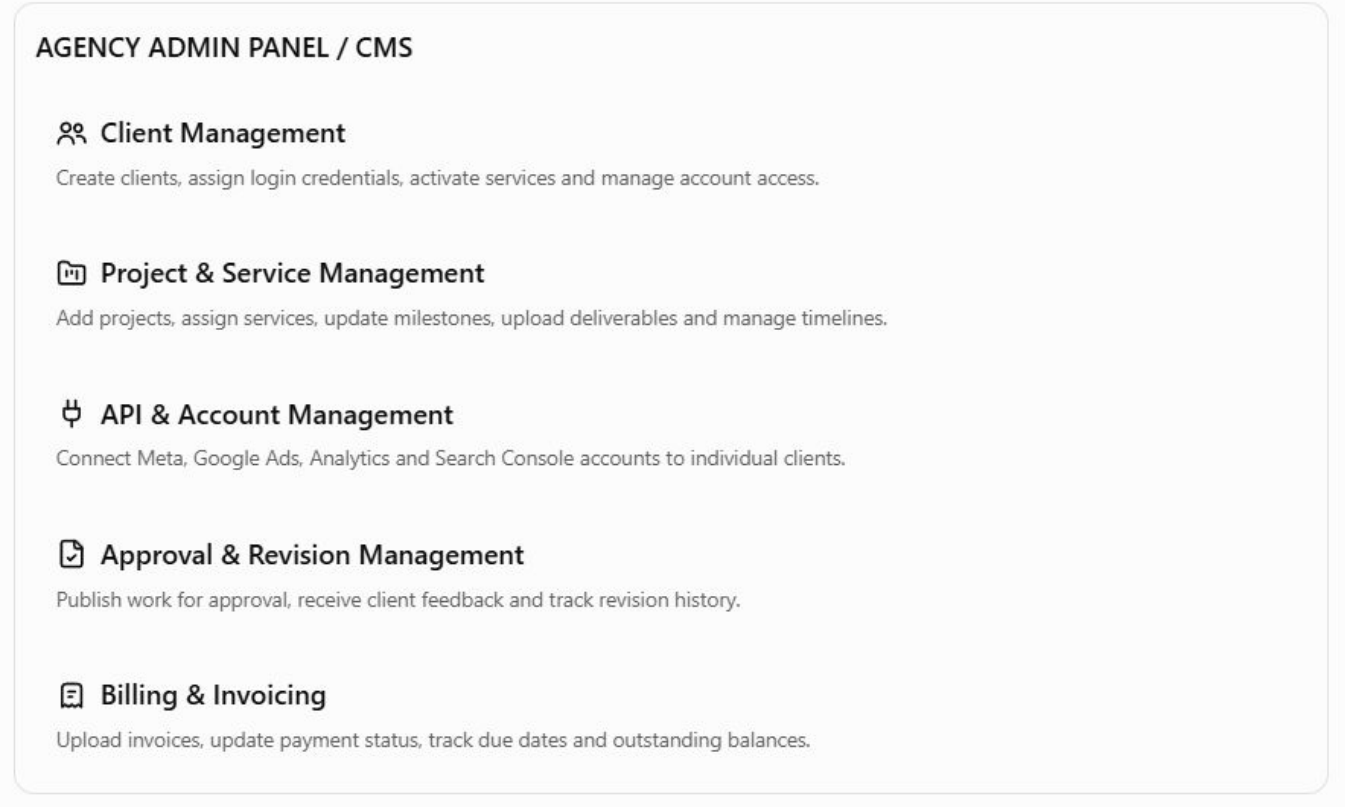
For example, if the client opens the dashboard at 11:00 AM, they might see:



This is the kind of experience we want to provide.

5. How the CMS should manage manual updates

Our CMS should have an independent administrative interface, where our internal team can manage every client and their subscribed services.



Example: How a manual update works

Let's say our team is handling a branding project for Client A.

1. Our designer uploads three logo concepts through the CMS.
2. The CMS administrator selects Client A and the Branding service.
3. The deliverable is published with the status Pending Client Approval.
4. Client A receives a notification.
5. The client logs into the portal and views the three concepts.
6. The client either approves the work or requests revisions with comments.
7. The response is recorded in the CMS.
8. Our team receives a notification and proceeds accordingly.

The same logic applies to website development milestones, creative production, campaign planning and other manual workflows.